

SAPIENZA
UNIVERSITÀ DI ROMA

DEPARTMENT OF COMPUTER, CONTROL AND MANAGEMENT ENGINEERING

Robot collector

INTERACTIVE GRAPHICS

Professor:
Marco Schaerf

Student:
Luca Del Signore
2096442

Academic Year 2025/2026

Contents

1 Introduction **3**

1.1 External imported libraries and models 3

2 Internal Kinematic Model of the UR5 robot **5**

2.1 URDF Mesh Frame Alignment 5

2.2 Inverse Kinematics of the complete model 6

2.2.1 Numerical Jacobian 6

2.2.2 Dumped Least Square 7

3 3D Scene Modeling **8**

3.1 Three.js Scene 8

3.2 Floor Mesh 8

3.3 Chest Mesh 8

3.4 Support Robot Base Block Mesh 9

3.5 Cube Mesh 9

3.6 UR5 Robot Mesh 9

3.7 Robotiq 85 Gripper Mesh 9

4 Physics Simulation **11**

4.1 Physics World 11

4.2 Rigid Bodies 12

4.2.1 Floor Rigid Body 12

4.2.2 Finger Contact Sensor Rigid Body 12

4.2.3 Cube Rigid Body 12

4.3 Synchronization Between Ammo.js and Three.js 13

4.4 Collision Detection 13

5 Robot Motion And Animation **14**

6 Model-View-Control design architecture **15**

6.1 Entity Subdivision 16

6.2 Controller/View of the Scene 17

6.3 Animation Loop 18

7 Application Flow **20**

7.1 Collect Cube use case 20

7.2 Release Cube use case 21

8 User Guide Interaction **22**

8.1 Use Case: Collect Cube 22

8.2 Use Case: Release Cube 23

8.3 Change Animation Period 24
8.4 Rotate Camera 24
8.5 Move Camera 24
8.6 Zoom In/Out 24
8.7 Interact with Objects 24
8.8 Reset Cubes 24

References **25**

1 Introduction

Robot Collector is a web-based virtual room simulator in which a UR5 manipulator is called by the user to grasp cubes scattered around the floor and place them in chests. Despite its simplicity, behind the scenes there is a really complex scenario:

- The kinematic internal model of the robot and the synchronization with the corresponding mesh;
- The composition of the 3D scene with meshes;
- The internal physics simulation;
- The robot motion animation;
- The engineering of the software through the MVC design pattern;
- The inverse kinematics to align the gripper with the selected cube;
- The collision system for detecting if the robot has a cube in its gripper.

Thus, this project is the combination of different subjects encountered during my study plan, and we are going to see all these technical aspects in detail.

1.1 External imported libraries and models

Being a web-based application, the programming language used is Javascript. Thus, the javascript libraries imported to help develop this project are:

- **Three.js** [1] for rendering and setting up the scene and the camera;
- **Ammo.js** [2] for collision detection between rigid bodies;
- **Tween.js**[3] for interpolating between two robots' configurations;
- **lil-gui** [4] for creating a UI Control Panel;
- **OrbitControls** [5] for rotating and moving the camera;
- **urdf-loader** [6] which exposes a practical API for loading and manipulating the URDF meshes;
- **ml-matrix** [7] for calculating the inverse of matrices

Regarding the meshes, the following visual models were imported:

- **UR5** robot from [8];
- **Robotiq 85 gripper** from [9].

Their structure is described using the **(Unified Robot Description Format)** format, which is an XML-based format that contains the informations of the robot's structure, geometry, joint and link frames, dynamic properties.
However, the structure described in these files, does not always coincide with the corresponding kinematic model, and thus there is the necessity of synchronizing them with some fixed rigid transformations, or otherwise the position and orientation measured in the Three.js world environment will not coincide with the pose computed by the internal kinematic model.

2 Internal Kinematic Model of the UR5 robot

The UR5 manipulator is a six degree-of-freedom robot, composed of only revolute joints. For the description of its kinematic model, the **Denavit–Hartenberg (DH)** convention was used. This convention exploits the natural **hierarchical structure** of the robot, allowing the transformation of each joint to be described relative to the previous one. In the official site of the Universal Robots (`https://www.universal-robots.com/articles/ur/application-installation/dh-parameters-for-calculations-of-kinematics-and-dynamics`) all the DH parameters of the UR5 are reported. The last frame of the kinematic chain is in the center of the flange. This is particularly useful for the subsequent step of attaching the gripper.

2.1 URDF Mesh Frame Alignment

Unfortunately, the frames used in the two URDF meshes (UR5 and gripper) have some misalignment w.r.t the internal kinematic model computed with Denavit–Hartenberg convention. Moreover, the frames defined in the URDF files do not match those of the Three.js environment, which uses a Y-up coordinate system. Therefore, some corrections and compensations were required to align the internal kinematic model with the corresponding mesh and with the Three.js coordinate system. The transformations applied to the UR5 are presented below in the order in which they were applied in the kinematic chain:

1. In the URDF file, the UR5 mesh's base_link_inertia frame has a rotation around z-axis, w.r.t the world frame, by π. Thus, a compensating fixed transformation is required to align the internal kinematic model (computed with DH matrices) with the robot mesh;

2. To pass from the Z-up coordinate system used by the DH parameters to the Y-up coordinate system used by Three.js, the mesh was rotated around x-axis of $-\frac{\pi}{2}$. The same transformation was applied to the internal kinematic model in order to maintain alignment with the mesh.

3. To place the robot mesh on the support robot base box mesh, a vertical translation was required. The same transformation was applied to the internal kinematic model in order to maintain alignment with the mesh.

Regarding the gripper, an additional fixed transformation must be applied before the previously described transformations and before combining it with the internal kinematic model. This transformation was required to correctly attach the gripper to the center of the flange (as the gripper origin is set at the center of its TCP). This homogeneous transformation matrix consists of a translation along the z-axis of 0.137 meters and a rotation around the y-axis of $-\frac{\pi}{2}$.

Now that the complete direct kinematics formulation can be extracted, the inverse kinematics of this structure can be defined.

2.2 Inverse Kinematics of the complete model

In this application, when an object is clicked, the pose of that object is easily recovered. Then, starting from that target pose, there is the necessity of finding which joints' configuration of the robot achieves that pose and make the gripper align with the selected object. In our case, the pose descriptor of the robot is given by the position of the TCP and its roll-pitch-yaw angles w.r.t. the Three.js world frame:

$$\mathbf{f}(\mathbf{q}) = \begin{bmatrix} x(\mathbf{q}) \\ y(\mathbf{q}) \\ z(\mathbf{q}) \\ \text{roll}(\mathbf{q}) \\ \text{pitch}(\mathbf{q}) \\ \text{yaw}(\mathbf{q}) \end{bmatrix}, \qquad \mathbf{q} = \begin{bmatrix} q_1 & q_2 & q_3 & q_4 & q_5 & q_6 \end{bmatrix}^T.$$

However, it is very difficult to find the inverses of these direct kinematics formulas in closed form. Therefore, it will now be presented the the numerical iterative algorithm used for computing the joints' configuration of the robot to reach the target pose.

2.2.1 Numerical Jacobian

Regardless of the chosen iterative algorithm, each of them uses the **Jacobian** of the direct kinematics formulas extracted from the homogeneous coordinates.

$$\mathbf{J}(\mathbf{q}) = \frac{\partial \mathbf{f}(\mathbf{q})}{\partial \mathbf{q}} = \begin{bmatrix} \frac{\partial x}{\partial q_1} & \frac{\partial x}{\partial q_2} & \frac{\partial x}{\partial q_3} & \frac{\partial x}{\partial q_4} & \frac{\partial x}{\partial q_5} & \frac{\partial x}{\partial q_6} \\ \frac{\partial y}{\partial q_1} & \frac{\partial y}{\partial q_2} & \frac{\partial y}{\partial q_3} & \frac{\partial y}{\partial q_4} & \frac{\partial y}{\partial q_5} & \frac{\partial y}{\partial q_6} \\ \frac{\partial z}{\partial q_1} & \frac{\partial z}{\partial q_2} & \frac{\partial z}{\partial q_3} & \frac{\partial z}{\partial q_4} & \frac{\partial z}{\partial q_5} & \frac{\partial z}{\partial q_6} \\ \frac{\partial \,\text{roll}}{\partial q_1} & \frac{\partial \,\text{roll}}{\partial q_2} & \frac{\partial \,\text{roll}}{\partial q_3} & \frac{\partial \,\text{roll}}{\partial q_4} & \frac{\partial \,\text{roll}}{\partial q_5} & \frac{\partial \,\text{roll}}{\partial q_6} \\ \frac{\partial \,\text{pitch}}{\partial q_1} & \frac{\partial \,\text{pitch}}{\partial q_2} & \frac{\partial \,\text{pitch}}{\partial q_3} & \frac{\partial \,\text{pitch}}{\partial q_4} & \frac{\partial \,\text{pitch}}{\partial q_5} & \frac{\partial \,\text{pitch}}{\partial q_6} \\ \frac{\partial \,\text{yaw}}{\partial q_1} & \frac{\partial \,\text{yaw}}{\partial q_2} & \frac{\partial \,\text{yaw}}{\partial q_3} & \frac{\partial \,\text{yaw}}{\partial q_4} & \frac{\partial \,\text{yaw}}{\partial q_5} & \frac{\partial \,\text{yaw}}{\partial q_6} \end{bmatrix}.$$

However, a numerical approach also for computing derivatives is necessary as the analytic Jacobian of our considered robot is really huge. So, each derivative in the Jacobian is approximated with:

$$\frac{\partial f_i}{\partial q_j} \approx \frac{f_i(q_1, \ldots, q_j + \varepsilon, \ldots, q_6) - f_i(q_1, \ldots, q_j - \varepsilon, \ldots, q_6)}{2\varepsilon}.$$

2.2.2 Dumped Least Square

The iterative inverse kinematics algorithm used to progressively update the joints' value towards the target pose is the **Dumped Least Square** method. It is a Newton-based Method that uses a damped **pseudoinverse** of the Jacobian (instead of using directly the inverse) to improve stability near singular configurations. Each iteration, the joints' update that reduces the error between the current gripper pose and the target gripper pose is computed. The process is repeated until the pose error falls below a predefined threshold or the maximum number of iterations is reached without convergence. The update step of the algorithm is:

$$\boxed{\mathbf{q}_{k+1} = \mathbf{q}_k + \mathbf{J}(\mathbf{q}_k)^T \left[\mathbf{J}(\mathbf{q}_k)\mathbf{J}(\mathbf{q}_k)^T + \lambda^2 \mathbf{I}\right]^{-1} \mathbf{e}_k}$$

where:

$$\mathbf{e}_k = \mathbf{pose}_{\text{target}} - \mathbf{f}(\mathbf{q}_k).$$

Then error vector can also be seen as

$$\mathbf{e} = \begin{bmatrix} e_x & e_y & e_z & e_{\text{roll}} & e_{\text{pitch}} & e_{\text{yaw}} \end{bmatrix}^T$$

The convergence is reached when:

$$\sqrt{e_x^2 + e_y^2 + e_z^2} < 10^{-4}$$

and at the same time:

$$\sqrt{e_{\text{roll}}^2 + e_{\text{pitch}}^2 + e_{\text{yaw}}^2} < 10^{-2}.$$

The convergence error threshold and the value of λ were determined empirically through several trials of the algorithm.

3 3D Scene Modeling

In this section we will examine in detail all the components of the 3D graphical environment that provides the workspace for the robot and how they were developed using **Three.js**, a JavaScript library for creating and rendering interactive 3D graphics in a web browser through **WebGL**.

3.1 Three.js Scene

The four most important ingredients provided by the Three.js API for creating a 3D graphical virtual environments are:

1. The `THREE.Scene` object, that is the object representing the virtual environment in which the other meshes are added. It organizes the meshes in a three structure so that if a parent mesh is moved, all the children meshes will move consequently;

2. The `THREE.PerspectiveCamera` object that permits the user to visualize the scene from a certain point of view;

3. The `THREE.WebGLRenderer` object that has the responsibility to draw on the screen the scene from the point of view of the camera;

4. The `THREE.DirectionalLight` object, because without a source of light the scene would be completely dark.

For our purpose, the camera is initialized in a position from which it is possible to visualize all the scene; however, the user has the possibility to change the position and orientation of the camera (as well as zoom in/out).
Then, for the same visualization purpose, four directional lights were placed above the four corners of the floor to illuminate the entire scene without producing any shadow.
Finally, the renderer continuously updates the drawn scene through an animation loop, which computes new poses accordingly to the application logic.

3.2 Floor Mesh

This is the base of the scene, where all the other objects are positioned. This object is a group of meshes containing all the floor tiles, that are 3D `THREE.BoxGeometry` spatially organized to obtain a visual checkerboard pattern.

3.3 Chest Mesh

The chest is an object composed of five `THREE.BoxGeometry` panel meshes, which are appropriately arranged in the space to obtain the final visual result.

3.4 Support Robot Base Block Mesh

There is a block in the center of the floor and that extends vertically, on which the robot is placed and it is a mesh created using `THREE.BoxGeometry` with height of 0.4 meters.

3.5 Cube Mesh

Each cube is a mesh created using `THREE.BoxGeometry` with the a width, height and depth of 0.047 meters. This size is small enough for the cube to fit between the gripper fingers. The cubes' poses in the scene are predefined and not randomly chosen, in order to ensure convergence of the inverse kinematics solver without encountering singular configurations.

3.6 UR5 Robot Mesh

The robot hierarchical model (as anticipated) is imported from an URDF file that describes the pose of each mesh w.r.t. the parent and in which folder the mesh and the relative texture is located. It contains also other informations regarding links and joints (for example the inertia of each link). However, in this project, only the kinematics data were considered.
The hierarchical model of the robot consists of six revolute joints:

1. shoulder pan;
2. shoulder lift;
3. elbow;
4. wrist 1;
5. wrist 2;
6. wrist 3.

Thanks to the API provided by [6], it is possible to set and access each of these joints by just one line of code: `this.ur5robot.setJointValue('name_of_the_joint', value)`. This simplifies the association between the joints' configuration computed by the internal hierarchical model and the visual mesh model.

3.7 Robotiq 85 Gripper Mesh

To make object manipulation more realistic, a **Robotiq 85 gripper** is attached to the robot flange.

The gripper is also imported from a URDF file and connected to the flange mesh of the UR5 manipulator, making it possible to exploit the hierarchical model for the gripper as well.
As mentioned before, a rigid transformation is applied to correctly position and orient the gripper mesh with respect to the robot flange mesh.
For the collision detection purposes, two additional invisible mesh volumes, created using `THREE.BoxGeometry`, are associated with the fingertips and overlap with them, as highlighted in red in figure 1. As will be discussed later, these simplified collision volumes are used to detect contact with the cube during collecting/releasing operations. Since they are part of the gripper, they move together with the entire hierarchical model. Their measured poses in the Three.js world are then used to update the corresponding sensor rigid bodies in the physics world simulation. In fact, they are moved only by the movement of the hierarchical model (**kinematic rigid bodies**) and not by gravity or collision.

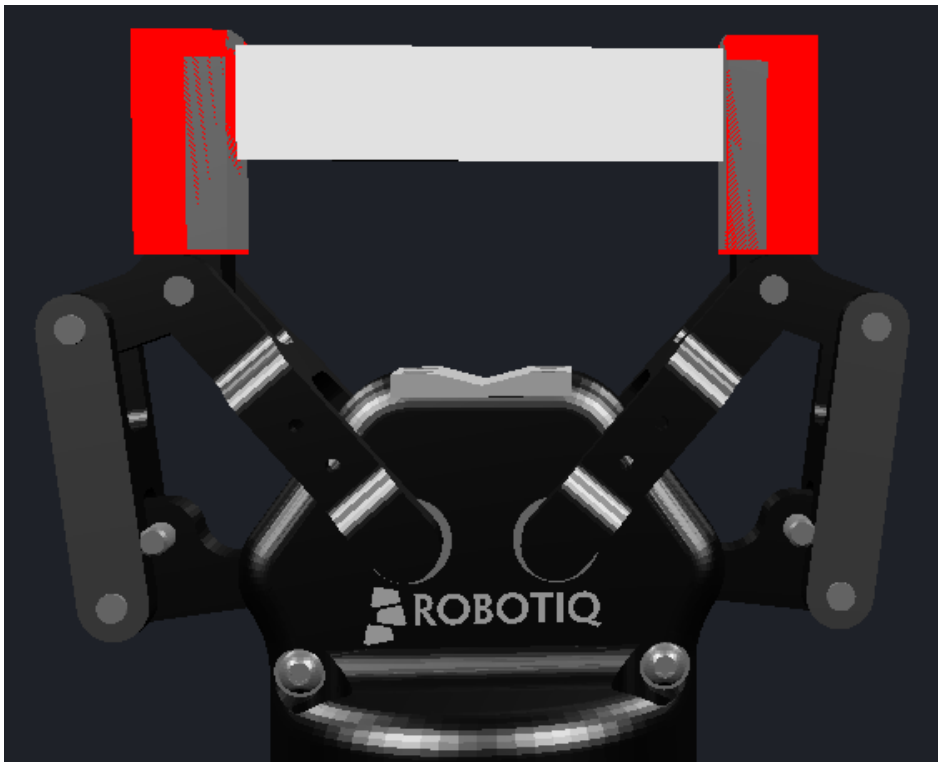


Figure 1: The two finger contact sensor meshes are highlighted in red.

4 Physics Simulation

The physics simulation was implemented using **Ammo.js**, which is a JavaScript port of the Bullet Physics engine. Its purpose is to provide realistic physical behaviour for the objects in the environment, including gravity, collision detection and rigid-body dynamics. The physics engine environment operates independently from the 3D scene modellation. In fact they are two completely distinct worlds, and both contain a separate representation of the same object. Then, synchronization between the two representations is required, and this happens during the animation loop. Three.js is responsible for the visualization of the objects, while Ammo.js manages the physical evolution of the objects.

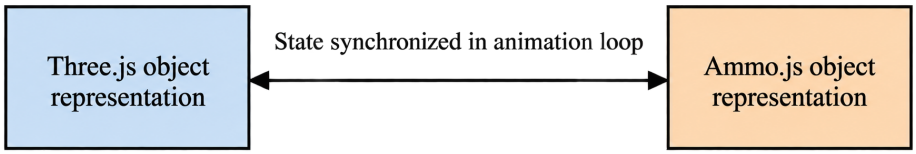


Figure 2: The relation between the Three.js and Ammo.js worlds

4.1 Physics World

At the beginning of the application, a physics world environment is created and configured with the required Ammo components, including the collision configuration, dispatcher, broadphase interface and constraint solver. From this point onward, the term **rigid body** refers to the associated physic volume in the Ammo world.
The physics world contains **static**, **kinematic** and **dynamic** rigid bodies. Static bodies are used for objects that are fixed in pose but can block the movement of the other objects ([`the floor`]); kinematic bodies are used for objects that are moved by the hierarchical movements of the robot but not by collisions with other rigid bodies or gravity ([`the two finger contact sensors`]); finally dynamic bodies are used for objects whose position and velocity are determined by the physics simulation. The cubes can behave either as dynamic or kinematic rigid bodies: when the gripper collects a cube, it becomes a kinematic rigid body moved only by the movement of the hierarchical robot; when it is not attached to the gripper, the cube is a dynamic rigid body that is affected by the gravity and by the collision with other rigid bodies.

4.2 Rigid Bodies

The most important features that define a rigid body in the Ammo physics environment are:

1. The shape of the rigid body;
2. The mass associated to the rigid body;
3. The pose occupied in the world.

While the first two features are fixed and set during the inizialization, the pose of the rigid body can vary during the use of the application with the modalities that will be explained in the 4.3 paragraph.

4.2.1 Floor Rigid Body

The floor in the Ammo environment is a static rigid body with a `BoxShape` collision shape, having the same dimensions and position of the floor mesh in Three.js. Since the floor must remain fixed throughout the simulation, its mass is set to zero. The graphical checkerboard visible in the scene is therefore associated with a simplified physical representation rather than physically modeling each individual graphical tiles. This approach avoids unnecessarily increasing the number of physical bodies while still providing a suitable collision surface for the dynamic rigid bodies.

4.2.2 Finger Contact Sensor Rigid Body

The two finger contact sensors in the Ammo environment are kinematic rigid bodies with a `BoxShape` collision shape, having the same dimensions and poses of the associated invisible meshes in the Three.js environment. Since they are kinematically moved by the robot and not by collision with other bodies in the environment, their mass is set to zero.

4.2.3 Cube Rigid Body

The cubes in the Ammo environment are dynamic rigid bodies with a `BoxShape` collision shape, having the same dimensions and poses of the associated meshes in the Three.js environment. To maintain this association, the same ID will be given to both mesh and rigid body representing the cube. The same approach is used also for the left/right finger contact sensors, which will have a fixed ID of 2000 and 3000 respectively.
Since the cubes must be affected by gravity and collisions with other rigid bodies, their mass is set to 1 kg. Moreover, when a cube is collected by the gripper, the application will set a flag to the corresponding rigid body so that it now becomes a kinematic rigid

body and it follows the transformation of the gripper rather than continuing to fall under gravity. When released, its dynamic properties are restored and the rigid body is allowed to interact with the physics simulation again (falling under gravity).

4.3 Synchronization Between Ammo.js and Three.js

The physics simulation is let evolve during the application's animation loop. After each physics evolution step, the transformations calculated by Ammo.js are retrieved from the dynamic rigid bodies and applied to their corresponding Three.js meshes. Then the renderer will draw the new situation.
Instead, for the contact sensors and for the grasped cube, in each animation step the poses are retrieved from the Three.js meshes and applied to their corresponding Ammo.js rigid bodies to inform the physics simulation that some rigid bodies have been set in a new pose, so the subsequent simulation steps must evolve from the updated configuration of the rigid bodies. This synchronization is necessary because the two libraries maintain a separate representation of the same physical object and then both the two environments (Three.js and Ammo.js) must exchange informations.

4.4 Collision Detection

Collision detection is used to determine if the robot's gripper is holding a cube. During the opening-closing animation of the gripper it is controlled, in the Ammo environment, if the associated rigid body of the selected cube is colliding both with the left-finger and right-finger rigid bodies. It is really simple to verify this collision because Ammo automatically calculates the points of contact between rigid bodies in its environment. Then, once that the two fingers' rigid bodies are touching the cube, the mesh of the cube is attached to the gripper hierarchical structure and its rigid body becomes a kinematic body.

5 Robot Motion And Animation

Once the inverse kinematics algorithm has computed a joints' configuration corresponding to the desired gripper pose, the initial and target joints' configurations must be interpolated into a smooth motion of the robot.
For this purpose, this application uses **Tween.js** library. It permits to define a transition between an initial set of values to a target set of values in a specified duration and with a specific easing function. It is updated inside the the continuous animation loop of the application, so that each frame produces a new intermediate robot configuration.
In this application, a sinusoidal easing function is used in order to have a smoother start and stop movement.
There are two tweens in this application: the tween described above that manages the animation-interpolation of the robot, and the tween that manages the animation-interpolation of opening and closing the gripper. Moreover, during this tween, it is also controlled if the gripper's contact sensors are both colliding with the selected cube. If the robot wants to collect the cube and both the two sensors detect collision with that cube, the animation-interpolation of that tween is interrupted as reached the situation in which the robot is firmly holding the cube.
Moreover, the animation duration period can be modified through the provided `User Control Panel`. When the duration is changed during an ongoing executing animation, the current state of the robot is preserved and the remaining motion is re-scaled proportionally to the animation period set.

6 Model-View-Control design architecture

During the development of this application, the necessity of reorganizing the code in modular subparts was born. Then, **Object-Oriented Programming (OOP)** was used in order to divide the responsibility of the various parts of the application (graphical rendering, application logic, physical simulation and state-machine management). Following the OOP paradigm, the code architecture chosen for this application is inspired to the **Model-View-Controller (MVC)** classic approach that perfectly aligns with the needs.
This is the subsequent subdivision:

- the Controller package, responsible for the application logic, user interaction state-machine, synchronization of the objects' state between the physical simulation (Ammo env.) and the graphical representation (Three.js scene);
- the View package, responsible for the Three.js graphical representation of the scene; it contains the meshes, lights, camera and other elements necessary for the visual representation of the scene;
- the Model package, responsible for storing the state of the scene and for the computation of direct and inverse kinematics; it contains the physical representation of objects in the Ammo env. (rigid bodies);

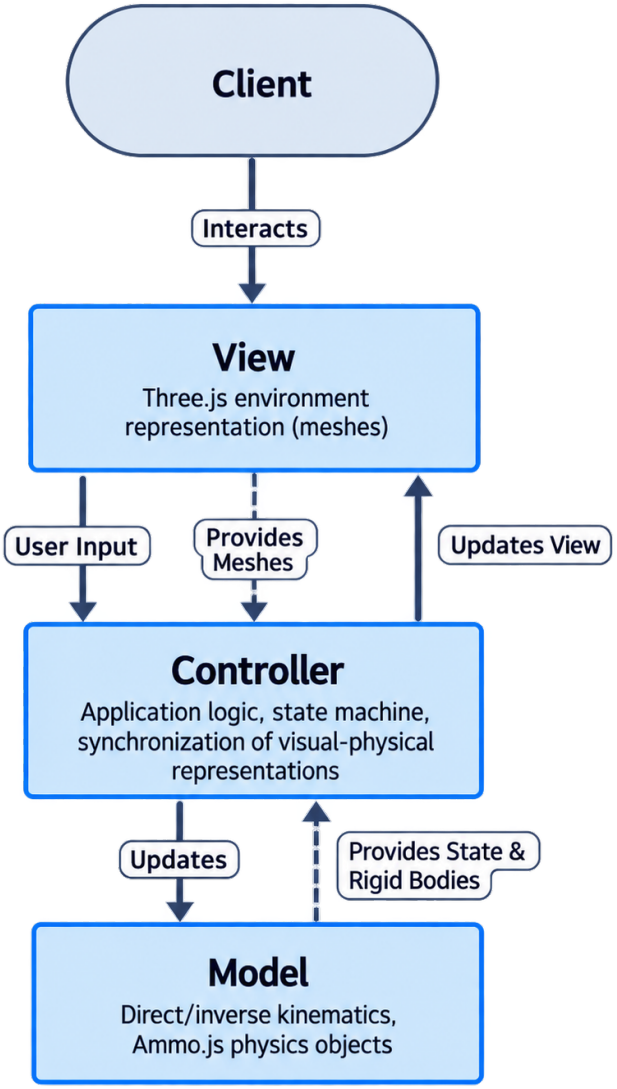


Figure 3: The MVC subdivision diagram

The Ammo physics world and the Three.js scene are treated as separate subsystems of the application, and they are recovered throughout the application using the singleton pattern.

6.1 Entity Subdivision

Each entity in the scene is represented by a set of Model, View, and Controller objects that decouple the responsibilities. The three components cooperate to manage the entity while keeping the physical rigid body representation, graphical representation, and application logic separated and synchronized according to the responsibilities described above. The entities present in the scene are **chests, the robot, the gripper, cubes, and the floor**. While the first four entities can have multiple instances in the scene, the floor is unique: only one floor can exist. For this reason, the floor Model, View, and Controller follow a Singleton pattern.

To illustrate how this representation works, the modeling of the entity "Cube" will be reported. The Controller of a particular cube gathers and manages the View and the Model instances of that cube in order to maintain the association and synchronization between them. For example, when a new cube is added to the environment, the controller is responsible for coordinating the creation of both the VCube and MCube (which, in turn, will create the corresponding mesh and rigid body respectively). This organization prevents the graphical and physical representations from becoming inconsistent. The figure below explains in detail this association and workflow:

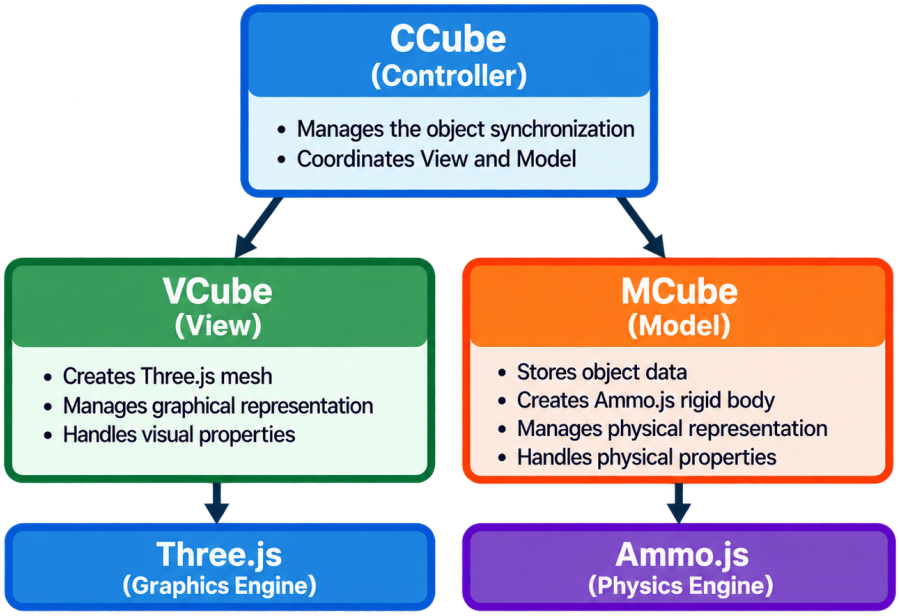


Figure 4: The MVC subdivision of the Cube entity.

For a detailed description of each package and class, see the JavaScript Doc in the JSDoc folder.

6.2 Controller/View of the Scene

All the described entities have a common upper-level singleton parent that manages and assorts all of them. The name given to this manager entity is **"Scene"**. Its responsibilities are divided between the View package and the Controller package. In this scenario, **VScene** setups the Three.js 3D visual environment with camera, lights and renderer, and provides an API to add new meshes to the scene.
Instead, **CScene** is the heart controller of the application:

- it implements the two use cases related to collecting and releasing cubes;

- it directly takes and manages user inputs;
- it ensures that all software objects remain coherent and synchronized by delegating specific tasks to the other controllers;
- it manages the animations tweens;
- it tracks the interaction state-machine of the application.

Regarding the last point, because several operations cannot be performed simultaneously, the application records an internal state machine to control the possible interactions, transitions and inputs.
For example, while the robot is moving towards a target, the user should not be able to start another robot movement. Once the movement is completed, the application transitions to a state in which interacting with objects in the scene is allowed.

6.3 Animation Loop

Another important responsibility delegated to **CScene** is the definition of the **Three.js animation loop**, which is executed once per frame. Within this loop, object instances are updated and then rendered on the screen. Here, the different subsystems (Three.js scene, Ammo.js engine and Tween.js interpolation) are coordinated:

- The tween engine will compute the new intermediate configuration of the ongoing tween animation interpolation (if a tween is currently running). This configuration is then propagated to the Model and View Packages of the robot and gripper, so that they can be effectively rendered in the new intermediate position. Moreover, the active tween also manages the controls variables that describes the interaction-state-machine;
- The Ammo.js physics world evolves accordingly to its internal physical state;
- After the physics simulation step, the visual representations of the cubes located on the floor or inside the chests are synchronized with their corresponding rigid bodies in the Ammo environment;
- If a particular cube is in the gripper of the robot, the current pose of that cube will be recovered from the visual hierarchical robot in Three.js and then propagated to the corresponding rigid body in the Ammo.js physics world;
- For the same reason, the poses of the finger contact sensors meshes will be recovered from the visual hierarchical robot in Three.js and then propagated to the corresponding rigid bodies in the Ammo.js physics world.

After this evolution process of the whole system and the relative synchronization between subsystems are completed, the renderer is called to draw the new scene.
A simplified execution flow is reported in the image below.

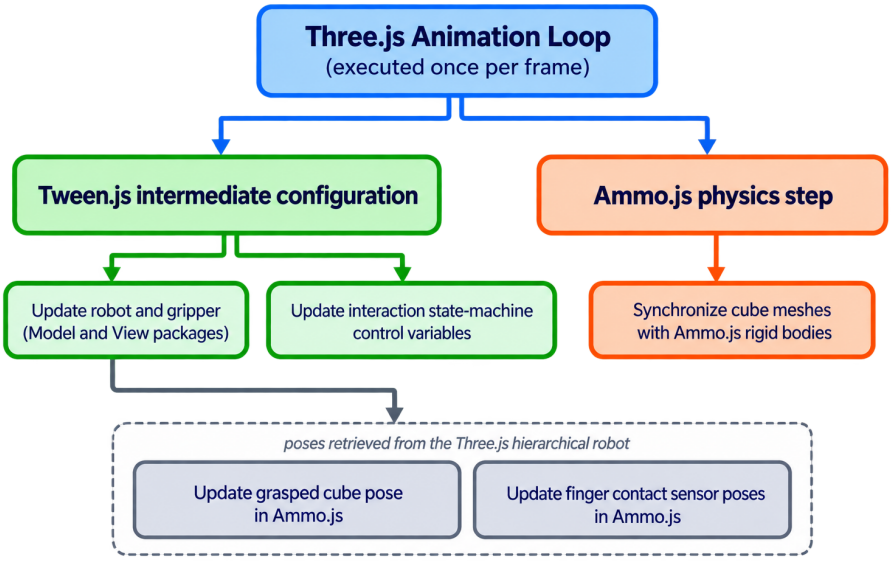


Figure 5: The Animation Loop update scenario

7 Application Flow

Now that we have all the ingredients, we will see in detail the steps of the two use cases of this application: the **Collect Cube use case** and the **Release Cube use case**. It is also reported, for each atomic step, the most important associated function for that task.

7.1 Collect Cube use case

Preconditions: The robot is not holding a cube and no animation is executing;
Main Flow:

1. The user will select (with the left click of the mouse) a cube mesh in the scene (green,cyan or white);

   [`CScene.register_mouse_clicked_input()`]

2. The application will verify if a cube was clicked using a ray cast from the camera through the clicked point on the screen panel;

   [`CScene.register_mouse_clicked_input()`]

3. If a cube is encountered by the ray, the application will recover, from the mesh, the pose and the ID of that cube. The ID will be used for recovering all the remaining cube informations from the associated controller;

   [`CScene.register_mouse_clicked_input()`]

4. The robot computes the inverse kinematics for two poses: the first is an intermediate pose with the same position and orientation as the cube, but with the y-coordinate increased by 0.2 m. The second is the real target pose of the cube. The robot first moves to the first intermediate pose to align itself above the cube. It then moves downward to the cube pose, placing the TCP in the cube center;

   [`CScene.collect_animation()`]

5. Then Tween.js will compute the midway joints' configuration to smoothly reach the intermediate pose first, then the target pose;

   [`CScene.move_Threejs_Robot()`]

6. Now that the TCP is aligned with the cube, the grasping animation will start (always managed through Tween.js). If, during the closing of the gripper, collisions are detected between the cube rigid body and both contact-sensor rigid bodies ([`CScene.open_close_Threejs_Gripper()`]), the grasp is considered successful and the cube mesh will be removed from the floor and attached to the robot hierarchical Three.js mesh model in the TCP as a kinematic rigid body ([`CScene.collect_cube()`]);

7. Finally the robot return to the rest joints' configuration using Tween.js interpolation.

   [`CScene.move_Threejs_Robot()`]

Postconditions: The robot is in the rest pose holding a cube

7.2 Release Cube use case

Preconditions: The robot is holding a cube and no animation is executing;
Main Flow:

1. The user will select (with the left click of the mouse) a chest mesh in the scene;

   [`CScene.register_mouse_clicked_input()`]

2. The application will verify if a chest was clicked using a ray cast from the camera through the clicked point on the screen panel;

   [`CScene.register_mouse_clicked_input()`]

3. If a chest is encountered by the ray, the application will recover from the mesh the pose and the ID of that chest;

   [`CScene.register_mouse_clicked_input()`]

4. The robot computes the inverse kinematics for the pose of the selected chest, but with the y-coordinate increased by 0.55 m. Then the robot moves to the target pose to align itself above the chest.

   [`CScene.release_animation()`]

5. Then Tween.js will compute the midway joints' configuration to smoothly reach the target pose;

   [`CScene.move_Threejs_Robot()`]

6. Now that the robot is aligned above the chest, the the gripper will start to open. If collisions between the cube rigid body and both contact-sensor rigid bodies are no longer detected ([`CScene.open_close_Threejs_Gripper()`]), the cube mesh will be released and removed from the robot hierarchical Three.js mesh model, and the cube rigid body will become again a dynamic body subject to gravity ([`CScene.release_cube()`]);

7. Finally the robot return to the rest joints' configuration using Tween.js interpolation.

   [`CScene.move_Threejs_Robot()`]

Postconditions: The robot is in the rest pose without holding the cube

8 User Guide Interaction

This final chapter is dedicated to the explanation of what the user can do in this application. All the user interactions are exposed and briefly described in the `User Control Panel` of the application 6; here, a more detailed description will be given.

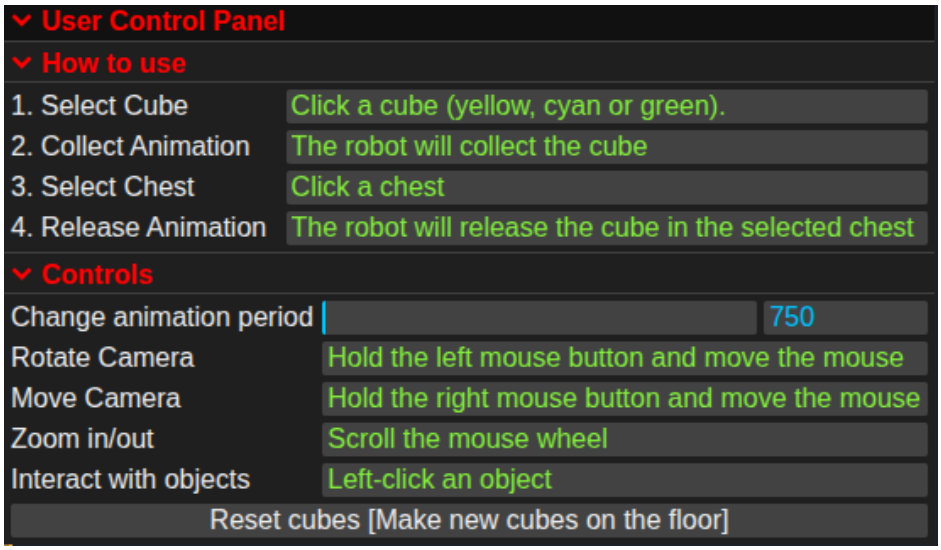


Figure 6: The `User Control Panel` in the upper-right part of the screen.

8.1 Use Case: Collect Cube

Initially, the robot is in its rest pose without any cube in its gripper. The user is asked to click and select one cube (cyan, yellow or green) on the floor as highlighted in figure 7. Then the robot will execute the collect animation to go where the cube is located, collect it and then return to the rest pose.
This use case cannot be activated if the robot already has a cube in its gripper or if the cube clicked is already in a chest.
Moreover, when executing an animation, the application will ignore every other object selection.

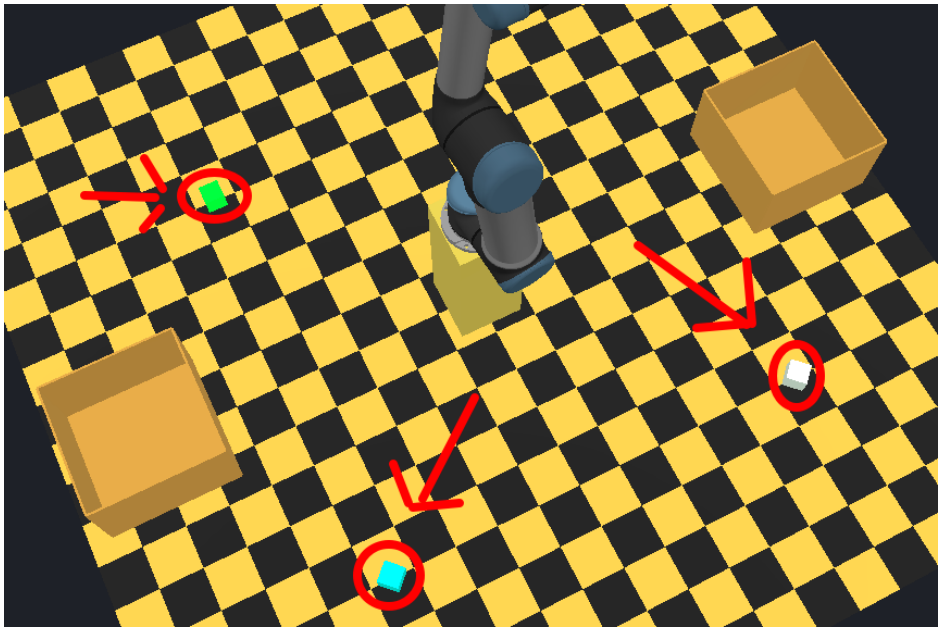


Figure 7: Select one of these three highlighted cubes

8.2 Use Case: Release Cube

Initially, the robot is in its rest pose with a cube in its gripper. The user is asked to click and select one chest to where release the cube as highlighted in figure 8. Then the robot will execute the release animation to go above the selected chest, release the cube in the chest and then return to the rest pose.
This use case can not be activated if the robot has not a cube in its gripper.
Moreover, when executing an animation, the application will ignore every other object selection.

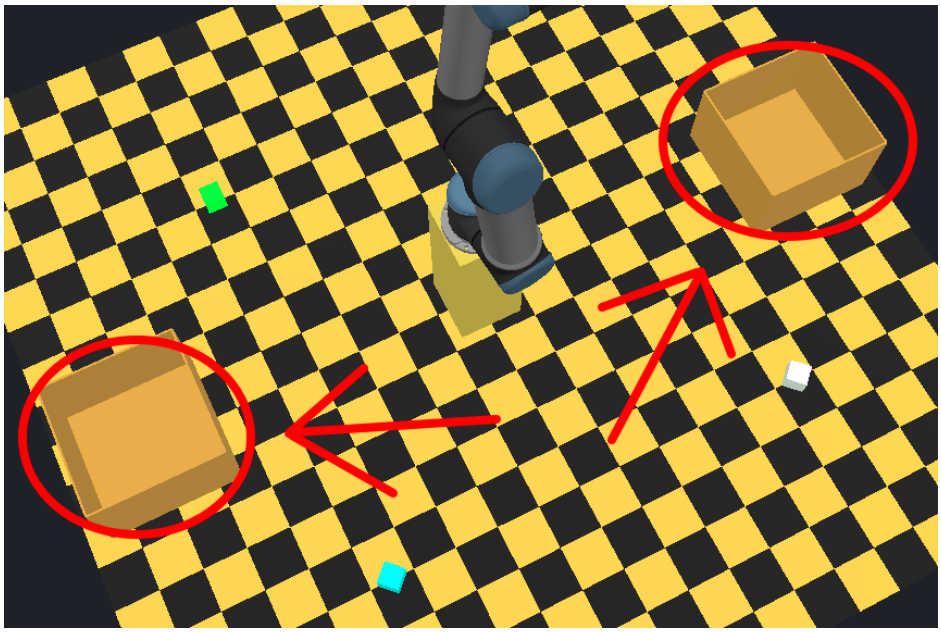


Figure 8: Select one of the two highlighted chests

8.3 Change Animation Period

The user can modify the velocity of animations by changing the value associated to the animation period (in milliseconds). This is also permitted when the robot is already executing an animation; in this case, the duration of the remaining part of the tween-animation will be re-scaled proportionally to the new animation period set.

8.4 Rotate Camera

By holding the left mouse button, it is possible to rotate the camera by dragging the mouse.

8.5 Move Camera

By holding the right mouse button, it is possible to move horizontally and vertically the camera by dragging the mouse.

8.6 Zoom In/Out

By scrolling the mouse wheel, it is possible to move the camera closer to or farther from the scene.

8.7 Interact with Objects

To interact with chests and cubes, the user can click the desired object using the left mouse button. However, as described above, the application may be in states in which interaction with a subset, or even all, of the objects in the scene is disabled.

8.8 Reset Cubes

When the are no cubes on the floor, or when the user wants to create additional cubes, it is possible to press the button "Reset Cubes". It will delete all the cubes that currently are on the floor and generate new cubes with the same pose and colors. However, it will not delete the cubes that are already in the chests or in the gripper of the robot.
When executing an animation, or when the robot has a cube in its gripper, the application will ignore this command.

References

[1] Three.js. https://threejs.org/.

[2] Ammo.js. https://github.com/kripken/ammo.js/.

[3] Tween.js. https://github.com/tweenjs/tween.js/.

[4] lil-gui. https://lil-gui.georgealways.com/#.

[5] OrbitControls. https://threejs.org/docs/pages/OrbitControls.html.

[6] URDF Loader Javascript Library. https://github.com/gkjohnson/urdf-loaders/tree/master.

[7] ml-matrix. https://github.com/mljs/matrix.

[8] Universal Robots ROS2 Description. https://github.com/UniversalRobots/Universal_Robots_ROS2_Description.

[9] pybullet_grasp_annotator_robotiq_85. https://github.com/matafela/pybullet_grasp_annotator_robotiq_85.